

Hands-On Tutorial: Running Your First Lex Program (Linux)

Course: 4CS501CC25 - Principles of Compiler Design — Lab Session **Goal:** Install Lex/Flex, write a simple lexical analyzer, compile it, and run it on a test input.

Part A: Install Lex and Yacc

Lex programs use the `.l` extension and need the **flex** (or **flex-old**) package. Yacc programs use the `.y` extension and need the **bison** package.

Step A1: Open the Terminal

What to do: Search for "Terminal" in your Linux search bar and open it. **Why:** All installation and compilation commands in this lab are typed into the terminal — there is no graphical installer for these tools. **Input:** None (just opening the application). **Output:** A blank terminal window with a command prompt, e.g. `user@ubuntu:~$`.

Step A2: Install Lex/Flex

What to do: Type the exact command:

```
bash
sudo apt-get install flex
```

Why: This downloads and installs the `flex` package (the modern, free implementation of Lex) from Ubuntu's package repository. **Input:** The command above; then type `y` and press Enter when asked `Do you want to continue? [Y/n]`. **Output:** Terminal text showing package download/installation progress, ending in a line like `Setting up flex (2.6.4-x) ...` with no error messages.

Step A3: Install Yacc/Bison

What to do: Type:

```
bash
sudo apt-get install bison
```

Why: `bison` is the GNU implementation of Yacc, used to generate parsers later in this course. Installing it now saves time in the next lab session. **Input:** The command above; type `y` to confirm. **Output:** Similar installation log ending with `Setting up bison (...) ...`.

Step A4: Verify the Installation

What to do: Type:

```
bash
flex --version
bison --version
```

Why: Confirms both tools are correctly installed and accessible before you start writing code — catching setup problems early rather than mid-program. **Input:** The two commands above. **Output:** Each command prints a version number, e.g.:

```
flex 2.6.4
bison (GNU Bison) 3.8.2
```

If instead you see `command not found`, the installation in Step A2/A3 did not complete — repeat that step.

Part B: Understand the Lex Program Structure

Every Lex source file (`.l`) has three sections, separated by `%%`:

```
%{
    // Declarations: #include statements, variable/function declarations
}%

%%
    // Translation rules: RegularExpression { C action }
%%

// User subroutines (e.g., main())
```

- `lex`/`flex` reads your `.l` file and generates a C file called `lex.yy.c`.
- You then compile `lex.yy.c` with `gcc` to produce an executable (`a.out`).
- Running `a.out` on an input stream tokenizes it according to your rules.

Flow: `mylex.l` → `(flex)` → `lex.yy.c` → `(gcc)` → `a.out` → tokens

Part C: Write Your First Lex Program

This program recognizes **integer constants**, discards newlines, and flags anything else as an invalid token.

Step C1: Create a Working Folder

What to do:

```
bash

mkdir ~/lex_lab
cd ~/lex_lab
```

Why: Keeps all your lab files (source, generated C code, executables, test inputs) organized in one place instead of scattered in your home directory. **Input:** The two commands above. **Output:** No visible output on success — your prompt now shows `~/lex_lab$`, confirming you're inside the new folder.

Step C2: Open a New Lex Source File

What to do:

```
bash
nano program1.1
```

Why: `nano` is a simple terminal text editor. The `.1` extension tells `flex` this file contains Lex specification code. **Input:** The command above. **Output:** An empty `nano` editing screen, ready for you to type code.

Step C3: Type the Lex Program

What to do: Type in the following code exactly:

```
c
%{
#include <stdio.h>
%}
%%
[0-9]+    { printf("%s is NUM\n", yytext); }
\n        { }
.         { printf("%s invalid token\n", yytext); }
%%
main()
{
    yylex();
}
```

Why (line-by-line explanation):

- `%{ #include <stdio.h> %}` — the **declarations section**; C code between `%{` and `%}` is copied verbatim into the generated file, here just including the standard I/O library for `printf`.
- `%%` — separates sections. The first `%%` marks the start of the **rules section**.
- `[0-9]+` — a regular expression matching **one or more digits**. When matched, the action `{ printf("%s is NUM\n", yytext); }` runs, printing the matched text tagged as `NUM`.
- `\n` — matches a newline character. The empty action `{ }` means it is simply discarded (no output), so line breaks don't clutter the token stream.
- `.` — matches **any single character** not already matched above (Lex tries rules in order and picks the longest/first match). This catches everything else — letters, symbols, spaces — and reports it as an `invalid token`.
- `yytext` — a built-in Lex variable that always holds the **exact text** just matched by the current rule.
- The second `%%` ends the rules section and begins the **user subroutines section**.
- `main() { yylex(); }` — your program's entry point. Calling `yylex()` starts the lexical analyzer, which keeps consuming input and applying the rules above until the input ends.

Input: The code block above, typed into the `(nano)` editor. **Output:** No terminal output yet — you're just authoring the source file.

Step C4: Save and Exit

What to do: Press `(Ctrl+O)`, then `(Enter)` to save, then `(Ctrl+X)` to exit `(nano)`. **Why:** Writes your typed code to disk as `(program1.1)` and returns you to the terminal prompt. **Input:** The key combinations above. **Output:** You're returned to the `(~/lex_lab$)` prompt. Running `(ls)` should now show `(program1.1)` in the folder.

Part D: Create a Test Input File

Step D1: Open a New Input File

What to do:

```
bash
nano test.c
```

Why: Your lexical analyzer needs some sample text to scan. This file plays the role of "source code" being fed into your analyzer — it doesn't need to be valid C, just a string of characters to tokenize. **Input:** The command above. **Output:** An empty `(nano)` editing screen.

Step D2: Type Sample Text

What to do: Type a simple line to test, for example:

```
a = 2 ;
```

Why: This line contains a mix of characters your rules will classify differently: a letter (`(a)`), spaces, an operator (`(=)`), a digit (`(2)`), and a semicolon (`(;)`) — so you can see both the `(NUM)` rule and the `(invalid token)` rule fire. **Input:** The line above. **Output:** No terminal output yet — just text being entered into the file.

Step D3: Save and Exit

What to do: Press `(Ctrl+O)`, then `(Enter)`, then `(Ctrl+X)`. **Why:** Saves the sample input as `(test.c)` in your working folder so it's ready to be used in Part E. **Input:** The key combinations above. **Output:** Returned to the `(~/lex_lab$)` prompt.

Part E: Compile and Run

Step E1: Generate the C Source from Your Lex Spec

What to do:

```
bash
flex program1.1
```

Why: `flex` reads your `.l` file and translates its rules into an equivalent C program that implements the scanning logic. **Input:** `program1.l` (the file you wrote in Part C). **Output:** No visible terminal message on success, but a new file `lex.yy.c` appears in your folder (check with `ls`). If there's a typo in your rules, `flex` will print an error here (e.g., `unrecognized rule`) instead of generating the file — see Part F.

Step E2: Compile the Generated C Code

What to do:

```
bash

gcc lex.yy.c -ll
```

Why: `lex.yy.c` is plain C code, so it needs a C compiler (`gcc`) to turn it into a runnable program. The `-ll` flag links the Lex runtime library, which provides supporting functions (like `yywrap`) that `lex.yy.c` depends on. **Input:** `lex.yy.c` (generated in Step E1). **Output:** No visible message on success. A new executable file `a.out` appears in your folder. If you see `undefined reference to 'yywrap'`, the `-ll` flag was missing — see Part F.

Step E3: Run the Analyzer on Your Test Input

What to do:

```
bash

./a.out < test.c
```

Why: `./a.out` runs your compiled lexical analyzer. The `<` symbol redirects the contents of `test.c` into the program as its input stream, exactly as if you had typed that text at the keyboard. **Input:** The contents of `test.c`, i.e. `a = 2 ;`. **Output:** Each character or token from `test.c` is classified by your rules and printed:

```
a is invalid token
= is invalid token
2 is NUM
; is invalid token
```

(Spaces are silently consumed by the `.` rule matching them one at a time, and the newline at the end of the line is discarded by the `\n` rule. `a`, `=`, and `;` don't match `[0-9]+`, so they fall through to the catch-all `.` rule and are reported as invalid; `2` matches `[0-9]+` and is correctly tagged `NUM`.)

Summary table:

Step	Command	Reads	Produces
E1	<code>flex program1.l</code>	<code>program1.l</code>	<code>lex.yy.c</code>
E2	<code>gcc lex.yy.c -ll</code>	<code>lex.yy.c</code>	<code>a.out</code>
E3	<code>./a.out < test.c</code>	<code>test.c</code>	Token output on screen

Part F: Common Errors and Fixes

Error	Cause	Fix
Unrecognized rule at line N	A regular expression overlaps with one defined earlier in the file	Reorder rules (more specific rules first) or fix the regex
undefined reference to 'yywrap'	The linker can't find yywrap()	Link with the Lex/Flex library: gcc program1.1 -ll (or -lfl for flex)
undefined reference to 'WinMain' (Windows only)	The OS expects a main() function	Add: int main() { yylex(); return 0; }

Part G: Try It Yourself

Once this works, extend the program to also recognize:

- Floating point numbers: `[0-9]*"."[0-9]+`
- Identifiers: `[_a-zA-Z][_a-zA-Z0-9]*`
- Operators: `[-+*/]`

Test it on an input like:

```
area = 22/7.0 * radius * radius
```

Expected token stream: `ID = NUM / FNUM * ID * ID`

Reference: Adapted from 4CS501CC25 - *Principles of Compiler Design Lab Session* slides and the Lex/Yacc Linux installation guide.